TUTORIAL DE GIT

Capítulo 5. Git en entornos distribuidos

Juan Felipe Castañeda Rodríguez · Fundación Universitaria Agraria de Colombia (UNIAGRARIA)
Ciencia de Datos — Actividad 2: Hands on, configuración de herramientas · Octubre de 2026

Una vez que existe un repositorio remoto, la pregunta es cómo organizar a las personas que contribuyen. Este capítulo presenta los flujos de trabajo distribuidos más comunes, cómo contribuir a un proyecto y cómo mantenerlo.

5.1 Flujos de trabajo distribuidos

Centralizado. Hay un repositorio central y todos empujan a él. Si alguien empujó antes, los demás deben traer y fusionar sus cambios antes de poder empujar. Es el flujo más simple y el que más se parece a Subversion; funciona muy bien en equipos pequeños.

Administrador de integración. Cada desarrollador tiene un repositorio público propio (un fork) además del oficial. Trabaja en su copia y pide al mantenedor que incorpore sus cambios. Es el modelo de GitHub con los pull requests.

Dictador y tenientes. Usado en proyectos enormes como el kernel de Linux: varios "tenientes" integran el trabajo de su área y un "dictador benevolente" fusiona el de los tenientes en el repositorio de referencia.

5.2 Contribuir a un proyecto

Pautas para confirmaciones:

- Revisar errores de espacios en blanco antes de confirmar:

```
git diff --check
```

- Hacer cada commit como un conjunto lógico de cambios; si se trabajó en varias cosas, dividirlas en varios commits (git add -p ayuda a preparar solo partes de un archivo).
- Escribir mensajes de commit claros: una línea corta de resumen, una línea en blanco y luego una explicación de la motivación del cambio.

Equipo pequeño privado. El flujo típico es:

```
git fetch origin
git merge origin/main
git push origin main
```

Si el push es rechazado porque alguien subió cambios antes, se repite: traer, fusionar y volver a empujar.

Equipo privado gestionado. Se trabaja en ramas por funcionalidad que se empujan al servidor; un integrador las fusiona en main.

Proyecto público con forks. Se hace un fork, se clona, se trabaja en una rama y se empuja al fork propio:

```
git clone https://github.com/mi_usuario/proyecto.git
git switch -c mejora
# ...cambios y commits...
git push -u origin mejora
```

```
git request-pull origin/main mi_fork
```

Luego se abre un pull request (en GitHub) o se envía el resumen al mantenedor.

Proyecto público por correo. Algunos proyectos reciben parches por correo:

```
git format-patch -M origin/main
git send-email *.patch
```

5.3 Mantener un proyecto

Quien mantiene un proyecto recibe contribuciones y debe integrarlas de forma ordenada.

Probar en ramas puntuales:

```
git switch -c sc/funcion_nueva main
```

Aplicar parches recibidos:

```
git apply --check parche.patch
git apply parche.patch
git am 0001-mejora.patch
```

Revisar ramas remotas de colaboradores:

```
git remote add colaborador https://github.com/colaborador/proyecto.git
git fetch colaborador
git log main..colaborador/mejora
git diff main...colaborador/mejora
```

La sintaxis de tres puntos compara contra el ancestro común, mostrando solo lo que aporta la rama.

Integrar: se puede fusionar directamente, usar ramas de integración (develop), hacer rebase o elegir commits individuales con:

```
git cherry-pick e43a6fd3
```

Marcar versiones y preparar entregas:

```
git tag -s v1.5 -m "Versión 1.5 firmada"
git describe main
git archive main --prefix="proyecto/" --format=zip > proyecto.zip
git shortlog --no-merges main --not v1.0
```

git shortlog genera un resumen de cambios por autor, útil para notas de versión.

5.4 Resumen

Git permite muchos modelos de colaboración: centralizado, con integrador o jerárquico. Contribuir bien implica commits pequeños, lógicos y bien descritos; mantener un proyecto implica revisar el trabajo en ramas, integrar con cuidado y etiquetar las versiones.

Referencia

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/es/v2